

Msc.CA Semester - 3
903-01 Server Side Scalable Application
Question Bank

MCQs:

1. Which command installs the yargs module in Node.js?

- a) npm install yargs
- b) npm add yargs
- c) npm get yargs
- d) npm use yargs

Answer: a) npm install yargs

2. What method in the fs module is used to synchronously write data to a file?

- a) fs.writeFile
- b) fs.writeFileSync
- c) fs.appendFileSync
- d) fs.createWriteStream

Answer: b) fs.writeFileSync

3. Which method from the fs module allows you to read a file asynchronously?

- a) fs.readFileSync
- b) fs.readFile
- c) fs.read
- d) fs.open

Answer: b) fs.readFile

4. How can you use fs.promises to write data to a file using the Promises API?

- a) fs.promises.createFile(file, data)
- b) fs.promises.writeFile(file, data)
- c) fs.promises.appendFile(file, data)
- d) fs.promises.saveFile(file, data)

Answer: b) fs.promises.writeFile(file, data)

5. Which method is used to get file information in Node.js?

- a) fs.fileStats()
- b) fs.stat()
- c) fs.fileInfo()
- d) fs.getStats()

Answer: b) fs.stat()

6. What does the EventEmitter class in Node.js facilitate?

- a) Handling file operations
- b) Managing asynchronous operations
- c) Creating and managing custom events
- d) Performing HTTP requests

Answer: c) Creating and managing custom events

7. Which method of EventEmitter registers a listener that is called only once for a specific event?

- a) on()
- b) once()
- c) addListener()
- d) emit()

Answer: b) once()

8. How do you start an application using PM2?

- a) pm2 start app.js
- b) pm2 run app.js
- c) pm2 launch app.js
- d) pm2 begin app.js

Answer: a) pm2 start app.js

9. Which method is used to remove an event listener in Node.js's EventEmitter?

- a) removeListener()
- b) removeEvent()
- c) detachListener()
- d) off()

Answer: a) removeListener()

10. Which of the following PM2 commands will show you the list of running applications?

- a) pm2 show
- b) pm2 list

- c) pm2 status
- d) pm2 running

Answer: b) pm2 list

11. What is callback chaining in Node.js?

- A. A technique for synchronizing asynchronous operations using promises
- B. A pattern to handle multiple asynchronous operations in sequence using callbacks
- C. A method for handling synchronous operations in a loop
- D. A way to handle synchronous file operations

Answer: B. A pattern to handle multiple asynchronous operations in sequence using callbacks

12. In the provided example of callback chaining, what is the purpose of fs.appendFile?

- A. To read content from a file
- B. To delete content from a file
- C. To write content to a file
- D. To append content to an existing file

Answer: D. To append content to an existing file

13. Which module provides promise-based file system operations in Node.js?

- A. fs
- B. fs.promises
- C. axios
- D. express

Answer: B. fs.promises

14. What does axios.get(url) do in the context of HTTP requests?

- A. It posts data to a URL.
- B. It retrieves data from a URL.
- C. It deletes data from a URL.
- D. It updates data at a URL.

Answer: B. It retrieves data from a URL.

15. How does express handle static files in an application?

- A. By using the `express.static` middleware
- B. By using the `fs.readFile` function
- C. By calling `app.use(express.static('public'))`
- D. By defining routes for each static file

Answer: C. By calling `app.use(express.static('public'))`

16. What is the purpose of `<%= %>` in an EJS template?

- A. To render unescaped HTML
- B. To output the result of a JavaScript expression, escaping HTML entities
- C. To execute JavaScript code without output
- D. To include partial templates

Answer: B. To output the result of a JavaScript expression, escaping HTML entities

17. When should `<%- %>` be used in an EJS template?

- A. To render escaped HTML
- B. To include partial templates
- C. To render raw HTML content without escaping
- D. To output the result of a JavaScript expression, escaping HTML entities

Answer: C. To render raw HTML content without escaping

18. How do you install EJS in a Node.js project?

- A. `npm install ejs`
- B. `npm install express`
- C. `npm install axios`
- D. `npm install fs`

Answer: A. `npm install ejs`

19. In the retail store example, what does the `/products/:id` route do?

- A. Displays the home page
- B. Displays a list of all products
- C. Displays the details of a single product based on its ID
- D. Adds a new product

Answer: C. Displays the details of a single product based on its ID

20. Which EJS tag should you use to include partials in a template?

- A. `<% include() %>`
- B. `<%= include() %>`
- C. `<%- include() %>`
- D. `<%- include('filename') %>`

Answer: D. `<%- include('filename') %>`

21. Which keyword is function-scoped in ES6?

- a) `let`
- b) `const`
- c) `var`
- d) `async`

Answer: c) `var`

22. Which of the following is true about the `let` keyword in ES6?

- a) It is function-scoped.
- b) It can be re-declared within the same scope.
- c) It is block-scoped.
- d) It must be initialized at the time of declaration.

Answer: c) It is block-scoped.

23. What will be the output of the following code?

```
const x = 5;  
  
x = 10;  
  
console.log(x);
```

- a) 5
- b) 10
- c) SyntaxError
- d) ReferenceError

Answer: c) SyntaxError

24. What does the await keyword do inside an async function?

- a) It pauses the execution of the entire program.
- b) It waits for a promise to resolve or reject.
- c) It makes a function synchronous.
- d) It immediately executes the function.

Answer: b) It waits for a promise to resolve or reject.

25. How do you handle multiple asynchronous operations concurrently using async and await?

- a) Use Promise.all()
- b) Use Promise.race()
- c) Use setTimeout()
- d) Use nested await statements

Answer: a) Use Promise.all()

26. What is the purpose of Babel in a JavaScript project?

- a) It transpiles modern JavaScript into older versions for compatibility.
- b) It provides a runtime environment for executing JavaScript.
- c) It optimizes code for better performance.
- d) It manages dependencies and packages.

Answer: a) It transpiles modern JavaScript into older versions for compatibility.

27. How can you execute modern JavaScript code directly with Node.js using Babel?

- a) By using babel-node
- b) By using node with .mjs files
- c) By configuring Babel in babel.config.js
- d) By using the --experimental-modules flag with node

Answer: a) By using babel-node

28. What type of variable declaration is block-scoped and cannot be re-declared within the same scope?

- a) var
- b) let

- c) const
- d) async

Answer: b) let

29. How do you export a function as the default export in an ES6 module?

- a) export function myFunction() {}
- b) export default myFunction;
- c) module.exports = myFunction;
- d) export { myFunction as default };

Answer: b) export default myFunction;

30. What does the following code snippet do?

```
import subtract, { PI, add } from './math.js';
```

```
console.log(add(2, 3));
```

- a) Imports and logs the subtract function from math.js.
- b) Imports the PI constant and add function from math.js and logs the result of add(2, 3).
- c) Imports all exports from math.js as add.
- d) Logs PI from math.js.

Answer: b) Imports the PI constant and add function from math.js and logs the result of add(2, 3).

31. What is Socket.io primarily used for?

- A) File transfer
- **B) Real-time web applications**
- C) Static web pages
- D) Image processing

32. Which method in Socket.io is used for broadcasting events?

- A) emit()
- **B) broadcast()**
- C) send()
- D) connect()

33. To share a user's current location, which type of data is typically used?

- A) String
- B) Number
- **C) Object with latitude and longitude**
- D) Array of strings

34. **Event acknowledgments in Socket.io are used for what purpose?**

- A) To send messages without a response
- **B) To confirm that an event has been received**
- C) To store user data
- D) To log errors

35. **Which state indicates a button is ready for interaction?**

- A) Disabled
- B) Loading
- **C) Active**
- D) Inactive

36. **How can timestamps be useful in a chat application?**

- A) For rendering user lists
- **B) For determining the order of messages**
- C) For sending messages
- D) For styling messages

37. **What is the purpose of tracking users joining and leaving in a chat application?**

- A) To improve UI design
- B) To manage user sessions
- C) To enhance security
- **D) To display online/offline status**

38. **Which Redis feature allows for efficient storage and retrieval of user data?**

- A) Lists
- B) Sets
- **C) Hashes**
- D) Strings

39. What is a typical use case for deploying a chat application?

- A) Single-user applications
- B) E-commerce platforms
- **C) Real-time customer support**
- D) Static content websites

40. Which component is NOT typically part of a chat application's architecture?

- A) Web server
- B) Database
- **C) Blockchain**
- D) Socket server

41. What is Mocha primarily used for?

- A) Database management
- B) Testing JavaScript applications
- C) Web development
- D) UI design

Answer: B) Testing JavaScript applications

42. Which of the following is a common assertion library used with Mocha?

- A) Jest
- B) Chai
- C) Jasmine
- D) Sinon

Answer: B) Chai

43. **How do you test asynchronous code in Mocha?**

- A) Using done() callback
- B) Using promises only
- C) By ignoring async functions
- D) By using global variables

Answer: A) Using done() callback

44. **What is the purpose of setup and teardown in testing?**

- A) To optimize performance
- B) To initialize and clean up test environments
- C) To compile code
- D) To mock external services

Answer: B) To initialize and clean up test environments

45. **When testing an Express application, which method is often used to simulate HTTP requests?**

- A) request()
- B) supertest
- C) fetch()
- D) axios

Answer: B) supertest

46. **What does chai.js provide in the context of testing?**

- A) Mocking framework
- B) Assertions and BDD/TDD styles
- C) Database connections
- D) API routing

Answer: B) Assertions and BDD/TDD styles

47. **Which assertion would you use to check for deep equality in Chai?**

- A) expect().to.equal()
- B) expect().to.deep.equal()

- C) assert.equal()
- D) should.equal()

Answer: B) expect().to.deep.equal()

48. In Mocha, which hook is used to run code before all tests in a suite?

- A) after()
- B) before()
- C) beforeEach()
- D) afterEach()

Answer: B) before()

49. What is the main use of mocking libraries in testing?

- A) To enhance performance
- B) To create fake objects for testing
- C) To manage dependencies
- D) To compile code

Answer: B) To create fake objects for testing

50. When wrapping up user tests, what is a key consideration?

- A) Code efficiency
- B) User experience
- C) Comprehensive coverage of use cases
- D) Database integrity

Answer: C) Comprehensive coverage of use cases

Short Questions:

1. What does the yargs module do in Node.js?
2. How do you install the yargs module?
3. How do you create a command using yargs?
4. What is the purpose of the fs module in Node.js?
5. How do you write to a file using fs synchronously and asynchronously?
6. How do you read a file using fs synchronously and asynchronously?

7. How do you open a file with fs and what is a file descriptor?
8. What are the steps to write to a file using fs with a file descriptor?
9. How do you read data from a file using fs with a file descriptor?
10. What is the syntax for using promises with the fs module?
11. How do you get file information using fs?
12. How do you close a file using fs?
13. What is the syntax for truncating a file with fs?
14. How do you delete a file using fs?
15. What is the syntax for creating a directory with fs?
16. How do you read the contents of a directory using fs?
17. How do you remove a directory with fs?
18. What is the purpose of the EventEmitter class in Node.js?
19. How do you create and use a custom EventEmitter instance?
20. How do you register and emit events using EventEmitter?
21. How can you ensure an event listener is triggered only once with EventEmitter?
22. How do you remove an event listener using EventEmitter?
23. What is PM2 used for in Node.js?
24. How do you install PM2 globally?
25. What command do you use to start an application with PM2?
26. How can you view the logs of applications managed by PM2?
27. What is callback chaining in Node.js?
28. How does callback chaining ensure the execution of asynchronous operations in order?
29. What is the purpose of the fs.readFile method in the callback chaining example?
30. How does Promise chaining differ from callback chaining in Node.js?
31. What is the main advantage of using promises over callbacks for handling asynchronous operations?
32. How do you handle errors in a promise chain?
33. What does the axios library do in Node.js applications?
34. How can you check if an HTTP request error is due to no response received?
35. What is Express used for in Node.js?
36. How do you set EJS as the templating engine in an Express application?
37. What is the difference between <%= %> and <%- %> in EJS?
38. When would you use <%- %> in an EJS template?
39. How do you include a partial template in EJS?
40. In an EJS template, how would you render a list of items using a loop?
41. What is the purpose of the views/partials/footer.ejs file in the retail store example?
42. What are the three ways to declare variables in JavaScript as introduced by ES6?
43. How does the let keyword differ from var in terms of scope?
44. Can you re-declare a variable declared with let within the same scope?
45. What happens if you try to update a constant declared with const?
46. How do arrow functions differ from traditional functions in JavaScript?
47. What is the syntax for a basic arrow function that adds two numbers?

48. How do you export a default function from a module in ES6?
49. How can you import a default export and a named export from a module?
50. What file extension is used for ES6 modules in Node.js when not using "type": "module" in package.json?
51. What is the purpose of the async keyword in JavaScript?
52. How do you handle errors in an async function?
53. What does the await keyword do in an async function?
54. How can you handle multiple asynchronous operations concurrently using async and await?
55. What is Babel used for in a JavaScript project?
56. What is the purpose of the .babelrc file in a project using Babel?
57. Explain the role of Socket.io in real-time applications.
58. What is the significance of broadcasting events in a chat application?
59. Describe how to share a user's location using Socket.io.
60. What are event acknowledgments, and why are they important?
61. How can you indicate that a button is in a loading state?
62. Discuss the importance of timestamps in message rendering.
63. What are the best practices for rendering a user list in a chat application?
64. How can you track user activity (joining/leaving) in a real-time chat app?
65. What considerations should be made when deploying a chat application?
66. Briefly explain how Redis can be used to manage user sessions.
67. Explain the role of Mocha in JavaScript testing.
68. What are assertions, and why are they important in testing?
69. Describe the difference between synchronous and asynchronous tests in Mocha.
70. How can you set up authentication tests in an Express application?
71. What is the purpose of using setup and teardown hooks in testing?
72. Outline the key features of chai.js.
73. How do you test an API endpoint using Mocha and supertest?
74. What are advanced assertions, and how do they differ from basic assertions?
75. Discuss the importance of mocking libraries in unit testing.
76. What steps are involved in setting up a test suite for a task application?

Long Questions:

1. What is the yargs module used for in Node.js? Explain how to install it and provide an example of using it to create a command-line command for adding two numbers.
2. What is the difference between synchronous and asynchronous methods in the Node.js fs module?
3. How do fs.readFileSync and fs.readFile work for reading files in Node.js? Provide a program example that reads a file using both methods.

4. How do `fs.open`, `fs.write`, and `fs.close` methods work?
5. How does the `fs.promises` API differ from callbacks in Node.js? Write a program using `fs.promises.writeFile` and `fs.promises.readFile` to handle file operations.
6. How can you get file information using `fs.stat` in Node.js?
7. How do you create, read, and remove directories using Node.js `fs` module?
8. What is `EventEmitter` in Node.js? Show how to create a custom event emitter, register listeners, emit events, and use one-time listeners.
9. What is PM2 and its main features? Describe how to install PM2 and use commands to start, monitor, restart, and stop applications.
10. Compare synchronous and asynchronous file operations in Node.js.
11. Describe how callback chaining in Node.js works for sequentially handling multiple asynchronous tasks.
12. Explain the concept of promise chaining in Node.js.
13. Describe how to use the `axios` library for making HTTP requests in Node.js.
14. Detail the steps to set up a basic Express server, including installation, route definition, request handling, and error management. Include a code example.
15. Explain what EJS is and how it generates dynamic HTML in Node.js.
16. Compare `<%= %>` and `<%- %>` in EJS, focusing on HTML escaping and use cases
17. Describe how to use EJS partials to create reusable components like headers and footers in a Node.js app.
18. Explain how to manage dynamic content and use partials in EJS templates for a retail store.
19. Describe how to initialize and configure a Node.js project with Express and EJS for a retail store.
20. What are the differences between `var`, `let`, and `const` in ES6?
21. How do ES6 arrow functions differ from traditional functions?
22. How do ES6 modules handle importing and exporting?
23. What are the steps to set up Babel in a Node.js project?
24. How do `async` and `await` simplify asynchronous operations in Node.js?
25. What is the process to create an API using ES6 and Node.js?

26. How does error handling with `async/await` compare to `promise .catch()`?
27. What are the benefits of using `Promise.all` for concurrent operations?
28. How do Babel's presets and plugins transform modern JavaScript features?
29. How do you implement a CRUD API with Express and ES6?
30. Discuss the architecture of a chat application utilizing Socket.io, including its components and their interactions.
31. Explain how you would implement broadcasting events in a chat application, including code examples.
32. Describe the process of sharing a user's current location within a chat application and the challenges involved.
33. Detail how event acknowledgments work in Socket.io and provide an example of their implementation in a chat app.
34. Analyze the different states of buttons in a web application and their impact on user experience.
35. Explain the significance of timestamps in a chat application, focusing on their role in message ordering and user experience.
36. Describe the methods for storing users in a chat application and rendering the user list efficiently.
37. Discuss the methods for tracking users joining and leaving a chat application, including potential data structures.
38. Outline the steps required to deploy a chat application and the considerations for ensuring scalability and performance.
39. Evaluate the role of Redis in building a chat application API, including its advantages and potential limitations.
40. Discuss the architecture of the Mocha testing framework and how it integrates with other libraries like Chai and Sinon.
41. Provide a detailed explanation of how to test asynchronous code using Mocha.
42. Explain how to structure tests for an Express application, including setup for routes and middleware testing.
43. Describe the process of testing a secured endpoint in an Express application with authentication.

44. Analyze the role of chai.js in enhancing testing capabilities in Mocha, detailing its syntax and features.
45. Explain advanced assertions in detail, providing examples of complex scenarios that may require them.
46. Evaluate the use of mocking libraries in unit testing and provide examples of how to mock dependencies in a Mocha test.
47. Outline the best practices for wrapping up user tests, focusing on coverage, maintainability, and performance.